Спецификация классов системы управления проектами и отслеживания задач

1. Пакет "Пользователи и Аутентификация"

1. Класс User (Пользователь)

Назначение: Представляет учетную запись пользователя в системе.

Атрибуты:

- id: Long - уникальный идентификатор пользователя
- firstName: String - имя пользователя
- lastName: String - фамилия пользователя
- email: String - электронная почта
- password: String - хэшированный пароль (bcrypt)
- status: String (текстовое представление текущего состояния)

Методы регистрации:

- register(name, email, password): Boolean
- validateRegistrationData(): Boolean
- retryWithNewData(newData): Boolean
- activate(activationToken): Boolean
- autoLogin(): String (возвращает sessionToken)

Методы аутентификации:

- login(email, password): Boolean
- logout(): void

Методы управления задачами:

- navigateToTasks(): void
- enterSearchKeywords(keywords): void
- clearSearch(): void
- applyFilters(filters): void
- resetFilters(): void
- selectTask(taskId): void
- backToList(): void
- edit(): void
- save(): void
- delete(): void

Методы профиля:

- updateProfile(profileData): Boolean

- saveProfile(): void
- cancelProfileEdit(): void

Методы управления аккаунтом:

- suspend(reason): void
- block(reason): void
- activate(): Boolean (реактивация)
- delete(): void (удаление аккаунта)

2. Класс ValidationsService (Сервис валидации)

Назначение: Проверяет данные пользователя на корректность.

Методы:

- validateEmailFormat(email: String): Boolean - проверяет формат email
- validatePasswordStrength(password: String): Boolean - проверяет сложность пароля

3. Класс UserRepository (Репозиторий пользователей)

Назначение: Обеспечивает доступ к данным пользователей.

Методы:

- save(user: User): Long - сохраняет пользователя и возвращает ID
- findByEmail(email: String): User - находит пользователя по email (возвращает null если не найден)

4. Класс RoleService (Сервис ролей)

Назначение: Управляет ролями пользователей.

Методы:

- assignDefaultRole(user: User, roleName: String):
  Boolean - назначает роль по умолчанию ("Executor")

5. Класс AuthService (Сервис аутентификации)

Назначение: Управляет аутентификацией.

Методы:

- autoLogin(userId: Long): String - выполняет автоматический вход и возвращает токен сессии

6. Класс RegistrationController

Назначение: Управляет процессом регистрации.

Методы:

- submitRegistration(userData): void - обрабатывает регистрацию
- registrationSuccess(sessionToken, user): void - обрабатывает успешную регистрацию

1. Класс RegistrationUI

Назначение: Пользовательский интерфейс для регистрации (может быть веб-страницей, мобильным экраном и т.д.)

Методы:

- submitRegistration(userData): void - отправляет данные регистрации в контроллер
- (На диаграмме не показаны другие методы UI, только это взаимодействие)

Класс Role (Роль)

Назначение: Определяет роль пользователя в системе и связанные с ней разрешения

Атрибуты:

- id: Long - уникальный идентификатор роли
- name: String - название роли (MANAGER, DEVELOPER, TESTER, ADMIN, CUSTOMER)
- permissions: List<Permission> - список разрешений, связанных с данной ролью

Методы:

- hasPermission(permissionCode): Boolean - проверяет наличие конкретного разрешения у роли

Класс Permission (Разрешение)

Назначение: Определяет конкретное действие, которое может выполнять пользователь

Атрибуты:

- id: Long - уникальный идентификатор разрешения
- code: String - код разрешения в формате "модуль:действие" (например: "task:create", "project:delete")
- description: String - подробное описание того, что позволяет делать разрешение

2. Пакет "Проекты и Задачи"

Класс Project (Проект)

Назначение: Представляет проект как основную единицу организации работ

Атрибуты:

- id: Long - уникальный идентификатор проекта
- name: String - название проекта, обязательное поле, максимальная длина 100 символов
- description: String - подробное описание проекта, максимальная длина 2000 символов
- startDate: Date - планируемая дата начала проекта

- endDate: Date - планируемая дата завершения проекта
- status: ProjectStatus - текущий статус проекта (PLANNING, ACTIVE, ON_HOLD, COMPLETED, CANCELLED)
- createdBy: User - пользователь, создавший проект
- createdAt: Date - дата и время создания проекта

Методы:

- createProject(projectData): Project - создает новый проект с указанными параметрами
- updateProject(projectData): Boolean - обновляет данные существующего проекта
- deleteProject(): Boolean - помечает проект как удаленный (soft delete)
- generateReport(reportType): Report - генерирует отчет по проекту указанного типа
- getTasks(statusFilter): List<Task> - возвращает список задач проекта с возможной фильтрацией по статусу

Класс Task (Задача)

Назначение: Представляет отдельную задачу в рамках проекта

Атрибуты:

- id: Long - уникальный идентификатор задачи
- title: String - заголовок задачи, обязательное поле, максимальная длина 200 символов
- description: String - подробное описание задачи, максимальная длина 5000 символов
- type: TaskType - тип задачи (FEATURE, BUG, IMPROVEMENT, DOCUMENTATION)
- priority: Priority - приоритет выполнения (LOW, MEDIUM, HIGH, CRITICAL)
- status: TaskStatus - текущий статус выполнения (TODO, IN_PROGRESS, CODE_REVIEW, TESTING, DONE)
- createdDate: Date - дата и время создания задачи
- dueDate: Date - срок выполнения задачи
- assignee: User - исполнитель задачи
- reporter: User - автор/создатель задачи
- project: Project - Фпроект, к которому относится задача
- sprint: Sprint - спринт, в который включена задача

Методы:

- createTask(taskData): Task - создает новую задачу с указанными параметрами
- updateTask(taskData): Boolean - обновляет данные существующей задачи

- changeStatus(newStatus, user): Boolean - изменяет статус задачи с записью в историю
- assignUser(user): Boolean - назначает исполнителя для задачи с проверкой загрузки
- getHistory(): List<TaskHistory> - возвращает полную историю изменений задачи

Класс Comment (Комментарий)

Назначение: Представляет комментарий к задаче как форму коммуникации между участниками проекта

Атрибуты:

• id: Long - уникальный идентификатор комментария

• task: Task - задача, к которой относится комментарий, обязательное поле

• author: User - пользователь, создавший комментарий, обязательное поле

• content: String - текст комментария, обязательное поле, максимальная длина 1000 символов

• createdDate: Date - дата и время создания комментария, обязательное поле

Методы:

• addComment(): Comment - создает новый комментарий с указанными параметрами

• deleteComment(): Boolean - помечает комментарий как удаленный (soft delete)

Класс TaskHistory (История задачи)

Назначение: Хранит историю всех изменений задачи для аудита и анализа

Атрибуты:

- id: Long - уникальный идентификатор записи истории
- task: Task - задача, к которой относится запись истории
- changedBy: User - пользователь, внесший изменение
- changeDate: Date - дата и время изменения
- fieldName: String - название измененного поля
- oldValue: String - предыдущее значение поля
- newValue: String - новое значение поля

Методы:

- recordChange(changeData): void - создает запись об изменении в истории

3. Пакет "Спринты"

Класс Sprint (Спринт)

Назначение: Представляет итерацию разработки в Agile-методологии

Атрибуты:

- id: Long - уникальный идентификатор спринта
- name: String - название спринта, обязательное поле
- startDate: Date - дата начала спринта
- endDate: Date - дата окончания спринта
- status: SprintStatus - текущий статус (PLANNED, ACTIVE, COMPLETED)
- project: Project - проект, к которому относится спринт

Методы:

- createSprint(sprintData): Sprint - создает новый спринт
- startSprint(): Boolean - запускает спринт (меняет статус на ACTIVE)
- completeSprint(): Boolean - завершает спринт (меняет статус на COMPLETED)
- addTask(task): Boolean - добавляет задачу в спринт
- removeTask(task): Boolean - удаляет задачу из спринта
- getTasks(): List<Task> - возвращает все задачи спринта

Класс Backlog (Бэклог)

Назначение: Хранит задачи, готовые к планированию в спринты

Атрибуты:

- id: Long - уникальный идентификатор бэклога
- project: Project - проект, к которому относится бэклог
- tasks: List<Task> - список задач в бэклоге

Методы:

- addTask(task, priority): Boolean - добавляет задачу в бэклог с указанным приоритетом
- removeTask(task): Boolean - удаляет задачу из бэклога
- getTasks(priorityFilter): List<Task> - возвращает задачи бэклога с фильтрацией по приоритету

4. Пакет "Отчеты"

Класс Report (Отчет)

Назначение: Представляет сгенерированный отчет по проекту или системе

Атрибуты:

- id: Long - уникальный идентификатор отчета
- title: String - название отчета
- type: ReportType - тип отчета (PROJECT_SUMMARY, SPRINT_PROGRESS, TEAM_PERFORMANCE, CUSTOM)
- generatedBy: User - пользователь, сгенерировавший отчет
- generationDate: Date - дата и время генерации отчета
- content: String - содержимое отчета в формате JSON или HTML

Методы:

- generate(parameters): Report - генерирует отчет на основе параметров
- exportToCSV(): File - экспортирует отчет в CSV формат
- exportToExcel(): File - экспортирует отчет в Excel формат
- display(): void - отображает отчет в веб-интерфейсе

Класс ReportGenerator (Генератор отчетов)

Назначение: Отвечает за создание различных типов отчетов

Атрибуты:

- reportTemplates: Map<String, Template> - шаблоны для различных типов отчетов

Методы:

- generateProjectReport(project, parameters): Report - генерирует отчет по проекту
- generateSprintReport(sprint, parameters): Report - генерирует отчет по спринту
- generateUserReport(user, parameters): Report - генерирует отчет по пользователю

5. Пакет "Уведомления"

Класс Notification (Уведомление)

Назначение: Представляет уведомление, отправляемое пользователю

Атрибуты:

- id: Long - уникальный идентификатор уведомления
- recipient: User - получатель уведомления
- title: String - заголовок уведомления
- message: String - текст уведомления
- type: NotificationType - тип уведомления (TASK_ASSIGNMENT, STATUS_CHANGE, SPRINT_START, DEADLINE_REMINDER, SYSTEM)
- sentDate: Date - дата и время отправки уведомления

- isRead: Boolean - флаг прочтения уведомления

Методы:

- send(): Boolean - отправляет уведомление получателю
- markAsRead(): Boolean - помечает уведомление как прочитанное

Класс NotificationService (Сервис уведомлений)

Назначение: Управляет отправкой и обработкой уведомлений

Атрибуты:

- emailService: EmailService - сервис для отправки email-уведомлений
- pushService: PushService - сервис для отправки push-уведомлений

Методы:

- sendTaskAssignment(task, assignee): Boolean - отправляет уведомление о назначении задачи
- sendStatusChange(task, oldStatus, newStatus): Boolean - отправляет уведомление об изменении статуса
- sendSprintStart(sprint): Boolean - отправляет уведомление о начале спринта

6. Пакет "Поиск и Фильтрация"

Класс SearchFilter (Фильтр поиска)

Назначение: Представляет критерии для поиска и фильтрации задач

Атрибуты:

- keywords: String - ключевые слова для полнотекстового поиска
- status: TaskStatus - фильтр по статусу задачи
- priority: Priority - фильтр по приоритету
- assignee: User - фильтр по исполнителю
- taskType: TaskType - фильтр по типу задачи
- project: Project - фильтр по проекту

Методы:

- apply(tasks): List<Task> - применяет фильтр к списку задач
- reset(): void - сбрасывает все установленные фильтры

Класс TaskSearchService (Сервис поиска задач)

Назначение: Обеспечивает расширенный поиск и фильтрацию задач

Атрибуты:

- index: SearchIndex - поисковый индекс для быстрого поиска

Методы:

- searchTasks(query, filters): List<Task> - выполняет поиск задач по запросу и фильтрам
- filterTasks(tasks, filters): List<Task> - фильтрует список задач по критериям
- buildSearchIndex(): void - перестраивает поисковый индекс

Перечисления (Enums)

TaskStatus

- TODO - задача создана, но не начата
- IN_PROGRESS - задача в работе
- CODE_REVIEW - задача на код-ревью
- TESTING - задача на тестировании
- DONE - задача завершена

Priority

- LOW - низкий приоритет
- MEDIUM - средний приоритет
- HIGH - высокий приоритет
- CRITICAL - критический приоритет (блокирующий)

TaskType

- FEATURE - новая функциональность
- BUG - исправление ошибки
- IMPROVEMENT - улучшение существующей функциональности
- DOCUMENTATION - работа с документацией

ProjectStatus

- PLANNING - планирование
- ACTIVE - активная разработка
- ON_HOLD - приостановлен
- COMPLETED - завершен
- CANCELLED – отменен

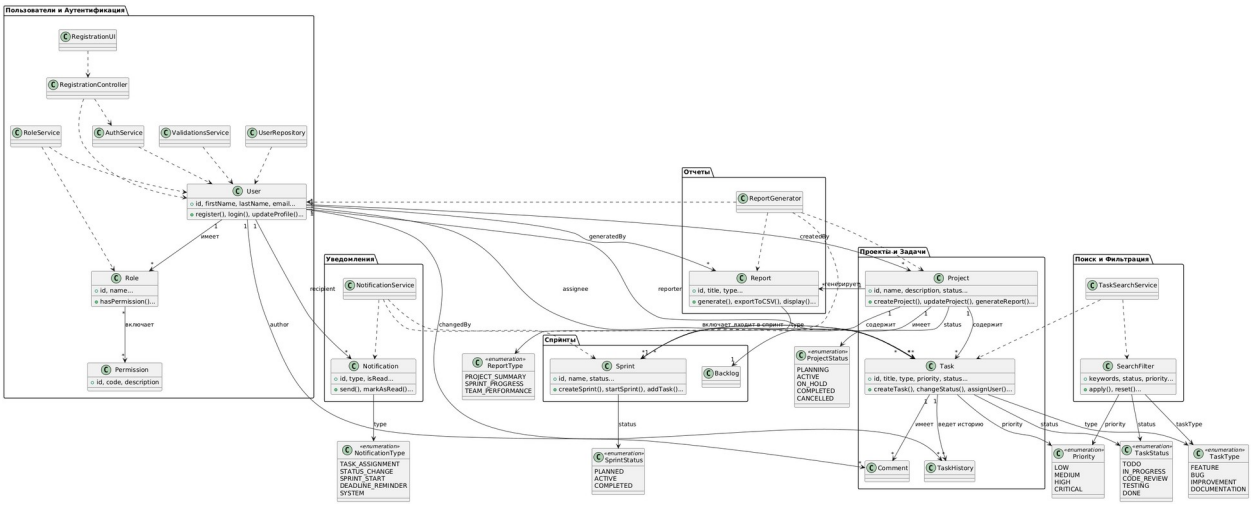
SprintStatus

- PLANNED - запланирован
- ACTIVE - активен
- COMPLETED - завершен

ReportType

- PROJECT_SUMMARY – сводка по проекту
- SPRINT_PROGRESS – прогресс спринта
- TEAM_PERFORMANCE – производительность команды

NotificationType

- TASK_ASSIGNMENT – назначение задачи

- STATUS_CHANGE – изменение статуса
- SPRINT_START – начало спринта
- DEADLINE_REMINDER – напоминание о дедлайне